Training Many LoRAs Together

A Distributional Optimisation Perspective on Combining Models in Deep Learning

---

Congye Wang

LAI Reading Group

Joint work with Yan Lin, Zheyang Shen, Matthew A. Fisher and Chris J. Oates
Newcastle University, UK

Roadmap

1. Large language models (LLMs)
2. How we fine-tune them: PEFT and LoRA
3. The research question: why **many** LoRAs?
4. Prior work: training LoRAs with MFLD
5. What MFLD does not do
6. Our method: VGD for many LoRAs
7. Results on eight commonsense benchmarks
8. A negative result: functional VGD
9. Future work

One-sentence summary

Treat a set of LoRA adapters as a **probability distribution**, and train them **jointly** by gradient descent in the space of distributions.

Large Language Models

A language model predicts the next token

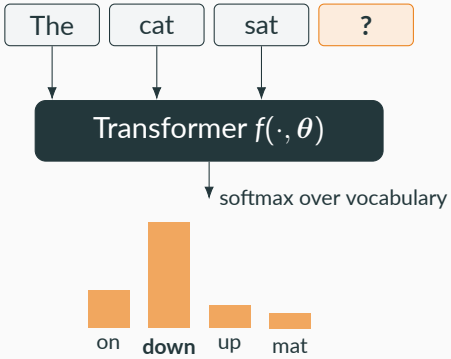
A language model factorises the probability of a token sequence:

$$p_\theta(y_{1:T}) = \prod_{t=1}^{T} p_\theta\big(y_t \mid y_{<t}\big).$$

Training minimises the next-token cross-entropy

$$L(\theta) = -\sum_t \log p_\theta(y_t \mid y_{<t}).$$

One simple objective + huge data + huge models $\Rightarrow$ many abilities appear “for free”.



The engine: the Transformer

Each layer has two blocks [Vaswani et al., 2017]:

- **Self-attention**

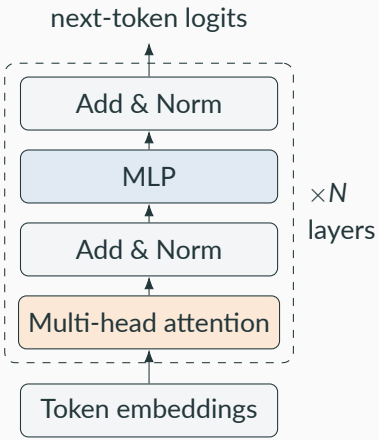
$$\mathrm{Attn}(\mathbf{X}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V},$$

  with $\mathbf{Q} = \mathbf{X}\mathbf{W}_q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_k$, $\mathbf{V} = \mathbf{X}\mathbf{W}_v$

- **MLP** (feed-forward) block

Almost all parameters live in **dense weight matrices** $\mathbf{W} \in \mathbb{R}^{d \times k}$.

This is exactly where LoRA will act.



Scale: why LLMs are both powerful and expensive

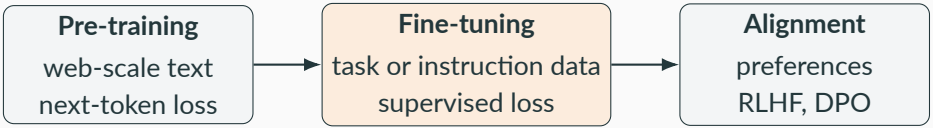
The base model: Llama-3-8B	
Parameters	8.0B
Layers	32
Hidden size	4096
Attention heads	32 (8 KV heads, GQA)

Also used: Llama-3.2-3B and Llama-3.2-1B.

Full fine-tuning is costly

- Adam keeps weights, gradients and two moment estimates
- Rule of thumb in mixed precision: $\approx$ 16 bytes per parameter
- 8B parameters $\Rightarrow$ well over **100 GB** of GPU memory, before activations
- One full copy of the model per task

The LLM life cycle



Key point

Pre-training is done once by a few labs. **Fine-tuning** is what most of us do: adapt a general model to a specific task with limited compute.

Fine-Tuning Large Language Models

Parameter-efficient fine-tuning (PEFT)

Freeze the base model. Train a **small** set of new parameters.

Family	Idea	Examples
Full fine-tuning	update every weight	—
Adapters	insert small bottleneck layers	Houlsby et al., 2019
Prompt-based	learn soft prompt / prefix vectors	Lester et al., 2021; Li & Liang, 2021
Selective	update a subset (e.g. biases)	BitFit, Ben Zaken et al., 2022
Low-rank	add a low-rank update to **W**	**LoRA**, Hu et al., 2022
LoRA variants	quantise, re-parametrise, adapt rank	QLoRA, DoRA, AdaLoRA

LoRA is the default choice in practice: simple, cheap and no extra inference latency.

LoRA: low-rank adaptation

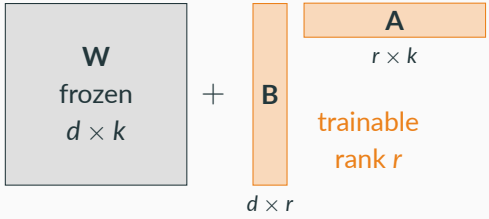
Replace a frozen weight $\mathbf{W} \in \mathbb{R}^{d \times k}$ by

$$\mathbf{W} + \Delta\mathbf{W}, \qquad \Delta\mathbf{W} = \mathbf{BA},$$

with $\mathbf{B} \in \mathbb{R}^{d \times r}$, $\mathbf{A} \in \mathbb{R}^{r \times k}$ and $r \ll \min\{d, k\}$.

- Only $(\mathbf{A}, \mathbf{B})$ are trained
- Init: $\mathbf{A}$ random, $\mathbf{B} = \mathbf{0} \Rightarrow \Delta\mathbf{W} = \mathbf{0}$
- Merge $\mathbf{W} \leftarrow \mathbf{W} + \mathbf{BA}$: **no extra latency**

Example: $d = k = 4096$, $r = 8$: 65,536 vs 16,777,216 numbers (0.39%).

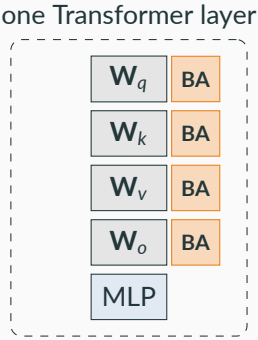


Where LoRA sits in a Transformer

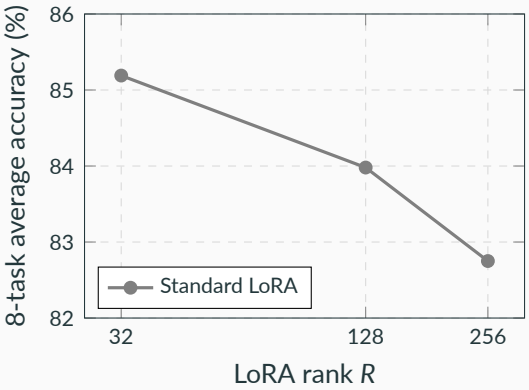
- Adapters go on selected matrices, e.g. $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v, \mathbf{W}_o$
- Each one has its own $(\mathbf{A}, \mathbf{B})$
- One base model, many small adapters

Notation for the rest of the talk

For one layer, write $\theta = (\mathbf{A}, \mathbf{B})$ for the adapter. Everything extends layer by layer.



A puzzle: a bigger rank does not help



Llama-3-8B, Commonsense-170K, single seed.

- In theory, a larger rank means more capacity
- In practice, standard LoRA gets **worse**: 85.19 $\rightarrow$ 82.75
- Optimising one big adapter is hard

Question

Can we get the extra capacity in a **different way**, by combining many small adapters?

The Research Question: Why Many LoRAs?

Two ways to combine m models

Ensemble (output averaging)

$$f(\mathbf{x}) = \sum_{i=1}^{m} w_i \, f(\mathbf{x}, \theta_i)$$

- Average the **outputs** (logits)
- More expressive than one model
- Test time: m forward passes

 $\Rightarrow$ ensemble of outputs

LoRA averaging

$$f(\mathbf{x}) = f\Big(\mathbf{x},\ \mathbf{W} + \sum_{i=1}^{m} w_i \, \mathbf{B}_i\mathbf{A}_i\Big)$$

- Average the **adapters**
- Combined rank at most $R = mr$
- Test time: **one** rank-R LoRA

 $\Rightarrow$ one merged adapter

Why many LoRAs?

1. **Capacity without the optimisation pain.**
   Many rank-r adapters give rank mr, but each piece is small and easy to train.
2. **Diversity helps.**
   Ensembles improve accuracy, robustness and uncertainty [Lakshminarayanan et al., 2017; Ovadia et al., 2019].
3. **It works empirically.**
   Merging 8 rank-32 adapters beats a single rank-256 adapter by about **4 points** (87.00 vs 82.75).
4. **Cheap at test time.**
   LoRA averaging merges into a single adapter: same cost as standard LoRA.

The real problem: how should we *train* them?

Common practice is **ad hoc**:

- Train each model independently (different seeds, data, hyperparameters)
- Combine afterwards with a fixed rule
- Training never "knows" the models will be combined
- Some models may get almost no weight: **wasted compute**

Research question

Is there a **principled objective** for training a set of models **jointly**, so that their **combination** performs well?

Key idea: lift to the space of distributions

Treat the set of models as a distribution Q over parameters, and solve

$$\boxed{\min_{Q \in \mathcal{P}(\mathbb{R}^p)} \mathcal{J}(Q) = \mathcal{L}(Q) + \lambda\, \mathcal{E}(Q)} \qquad \mathcal{E}(Q) = \int q \log q \;\; \text{(negative entropy)}$$

Ensemble loss

$$\mathcal{L}(Q) = \sum_{j=1}^{n} L\Big(\mathbf{y}_j,\ \int f(\mathbf{x}_j, \boldsymbol{\theta})\, \mathrm{d}Q(\boldsymbol{\theta})\Big)$$

LoRA-averaging loss

$$\mathcal{L}(Q) = \sum_{j=1}^{n} L\Big(\mathbf{y}_j,\ f\big(\mathbf{x}_j, \mathbf{W} + \int \mathbf{BA}\, \mathrm{d}Q\big)\Big)$$

- A set of m adapters is the particle approximation $Q_m = \frac{1}{m} \sum_{i=1}^{m} \delta_{\boldsymbol{\theta}_i}$
- Entropy ($\lambda > 0$) makes the problem well-posed: the minimiser of $\mathcal{L}$ alone has finite discrete support

Ensembles and LoRA averaging are *not* the same problem

Proposition (convexity)

If $\mathbf{u} \mapsto L(\mathbf{y}_j, \mathbf{u})$ is convex, then the ensemble loss $\mathcal{L}$ is convex on $\mathcal{P}(\mathbb{R}^p)$, and $\mathcal{J}$ is strictly convex for $\lambda > 0$.

- **Ensemble:** $Q \mapsto \int f(\mathbf{x}_j, \boldsymbol{\theta}) \, \mathrm{d}Q$ is affine, so convexity is inherited.
- **LoRA averaging:** $Q \mapsto \int \mathbf{BA} \, \mathrm{d}Q$ is affine, **but** $\mathbf{u} \mapsto L(\mathbf{y}_j, f(\mathbf{x}_j, \mathbf{W} + \mathbf{u}))$ is **not convex**, because the network is non-linear in its weights.

Consequence

Convergence theory that needs convexity holds for ensembles, but **does not transfer** to LoRA averaging.

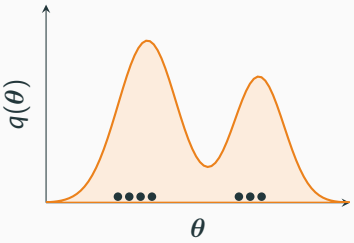
Prior Work: Training Many LoRAs with MFLD

Inspiration: mean-field neural networks

A one-hidden-layer network with m neurons:

$$f(\mathbf{x}) = \frac{1}{m} \sum_{i=1}^{m} \Phi(\mathbf{x}, \boldsymbol{\theta}_i) \longrightarrow \int \Phi(\mathbf{x}, \boldsymbol{\theta}) \, \mathrm{d}Q(\boldsymbol{\theta})$$

- Neurons = particles; the network = a distribution Q
- Training = optimisation over $Q \in \mathcal{P}(\mathbb{R}^p)$
- Gives global-convergence results for wide networks [Mei et al., 2018; Chizat & Bach, 2018]



particles approximate Q

Mean-field Langevin dynamics (MFLD)

MFLD is the McKean–Vlasov SDE [Hu et al., 2021; Chizat, 2022]

$$\mathrm{d}\boldsymbol{\theta}_t = -\nabla_V \mathcal{L}(Q_t)(\boldsymbol{\theta}_t)\,\mathrm{d}t + \sqrt{2\lambda}\,\mathrm{d}\mathbf{B}_t, \qquad Q_t = \mathrm{Law}(\boldsymbol{\theta}_t).$$

With m particles and Euler–Maruyama, step size ϵ:

$$\boldsymbol{\theta}_i^{t+1} = \boldsymbol{\theta}_i^t \;-\; \epsilon\,\nabla_V \mathcal{L}(Q_m^t)(\boldsymbol{\theta}_i^t) \;+\; \sqrt{2\lambda\epsilon}\,\mathbf{Z}_i^t, \qquad \mathbf{Z}_i^t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

- "Noisy SGD in the space of distributions"
- ∇_V is the variational gradient; on particles, $\nabla_V \mathcal{L}(Q_m)(\boldsymbol{\theta}_i) = m\,\nabla_{\boldsymbol{\theta}_i} \mathcal{L}(Q_m)$
- If $\mathcal{J}$ is convex: $Q_m^t \to Q^\star$ as $t \to \infty$, $m \to \infty$

Prior work: MFLD for LoRA [Nitanda et al., ICML 2025]

Propagation of chaos for MFLD and its application to model ensemble

- Each particle is a **rank-1** adapter $(\mathbf{a}_i, \mathbf{b}_i)$
- $m = 32$ particles form one adapter

$$\Delta \mathbf{W} = \frac{1}{m} \sum_{i=1}^{m} \mathbf{b}_i \mathbf{a}_i^\top, \quad \text{rank} \leq 32$$

- Trained with MFLD (noisy AdamW), $\lambda = 10^{-5}$
- Theory: finite-particle error bounds

PoC merge

1. Train M adapters **independently**, each rank $\leq R$
2. Merge: $\frac{1}{M} \sum_{i=1}^{M} \Delta \mathbf{W}_i$, rank $\leq MR$

Merged adapter has the test-time cost of one standard rank-MR LoRA.

What MFLD achieves

Llama-3-8B, 8-task average (%)

Method	Avg
Standard, rank 32	84.97
MFLD, rank 32	85.16
Standard, rank 256	82.75
PoC merge Standard, 8×32	87.00
PoC merge MFLD, 8×32	**87.14**

our reproduction, rank-32 rows: mean of 8 seeds

- MFLD gives a small gain over standard LoRA at rank 32
- The big gain comes from **merging many adapters**: about $+2$ points over a single rank-32 adapter
- Clear evidence that “many LoRAs” is a good idea

What MFLD Does Not Do

MFLD couples particles only implicitly

For LoRA averaging, the MFLD update of particle i is

$$\theta_i^{t+1} = \theta_i^t - \epsilon \underbrace{m \nabla_{\theta_i} \mathcal{L}(Q_m^t)}_{\text{gradient of the shared loss}} + \underbrace{\sqrt{2\lambda\epsilon}\, Z_i^t}_{\text{independent noise}}$$

- The only coupling is through the **shared prediction** $\mathbf{W} + \frac{1}{m}\sum_j \mathbf{B}_j\mathbf{A}_j$
- With $\lambda = 0$ this is just **standard LoRA training** of one rank-R adapter
- So MFLD = standard LoRA + isotropic Gaussian noise
- No particle ever **looks at** where the other particles are

In short

Diversity comes from **random noise**, not from **design**.

Three gaps in MFLD for many LoRAs

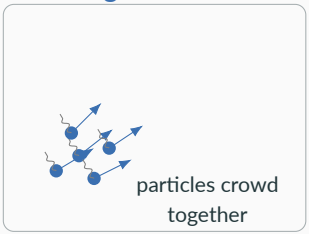
1. **No direct interaction.**
   Particles do not share gradient information and are not pushed apart on purpose.
2. **The merge is not joint.**
   In the PoC merge, the M adapters are trained in M separate runs and averaged afterwards.
3. **Theory does not apply.**
   MFLD guarantees need convexity of $\mathcal{J}$. LoRA averaging is non-convex.

Symptom at high rank

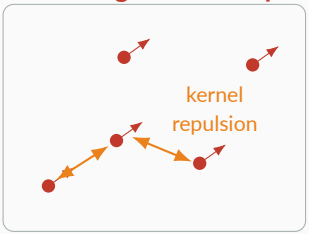
For a single rank-256 adapter, MFLD falls **below** standard LoRA: 81.83 vs 82.75 (paired difference -0.92, 95% CI $[-1.20, -0.63]$).

What we want instead

MFLD: gradient + noise



VGD: shared gradient + repulsion



- A **deterministic** method that optimises the same objective $\mathcal{J}(Q)$
- Particles **share** gradient information and **repel** each other
- Same cost order as MFLD

Our Method: VGD for Many LoRAs

Variational gradient descent (VGD)

Idea: deterministic steepest descent of $\mathcal{J}$ over distributions [Chazal et al. (2025)].

Move every particle by a small vector field $\mathbf{v}$: $Q \mapsto (\mathbf{I} + \epsilon\mathbf{v})_{\#}Q$. Then

$$\frac{\mathrm{d}}{\mathrm{d}\epsilon}\mathcal{J}\big((\mathbf{I} + \epsilon\mathbf{v})_{\#}Q\big)\Big|_{\epsilon=0} = \int \Big[\nabla_V \mathcal{L}(Q)(\boldsymbol{\theta}) \cdot \mathbf{v}(\boldsymbol{\theta}) - \lambda\,(\nabla \cdot \mathbf{v})(\boldsymbol{\theta})\Big]\mathrm{d}Q(\boldsymbol{\theta}).$$

- Choose $\mathbf{v}$ in the unit ball of an RKHS $\mathcal{H}_k^p$ with kernel k
- The steepest direction has a **closed form**
- Generalises Stein variational gradient descent [Liu & Wang, 2016] from linear $\mathcal{L}$ to general $\mathcal{L}$

The VGD update

$$\theta_i^{t+1} = \theta_i^t + \epsilon\,\phi(\theta_i^t), \qquad \phi(\theta_i) = \frac{1}{m}\sum_{j=1}^{m}\Big[\underbrace{-\,k(\theta_i,\theta_j)\,\nabla_V \mathcal{L}(Q_m)(\theta_j)}_{\text{kernel-smoothed gradient}} + \underbrace{\lambda\,\nabla_2 k(\theta_i,\theta_j)}_{\text{repulsion}}\Big]$$

Term 1: share information

Each particle moves using the gradients of its **neighbours**, weighted by similarity $k(\theta_i, \theta_j)$.

Term 2: keep diversity

For a Gaussian kernel, $\sum_j \nabla_2 k(\theta_i,\theta_j) = \frac{2}{\ell^2}\sum_j k(\theta_i,\theta_j)(\theta_i - \theta_j)$: pushes θ_i **away** from close particles.

Entropy is handled by **deterministic repulsion** instead of random noise.

VGD and MFLD: two views of the same objective

	MFLD	VGD
Objective	$\mathcal{L}(Q) + \lambda\mathcal{E}(Q)$	$\mathcal{L}(Q) + \lambda\mathcal{E}(Q)$
Dynamics	stochastic (SDE)	deterministic (ODE)
Entropy enters through	Gaussian noise	kernel repulsion
Particle interaction	only via shared loss	shared loss + kernel
Limit $\ell \to 0$ of VGD	—	MFLD without noise
Setting $\lambda = 0$	standard training	kernel-smoothed gradient
Convergence theory	needs convexity	[Chazal et al., 2025]

Note: VGD with $\lambda = 0$ is **not** standard fine-tuning. The kernel still couples the particles.

Making VGD work for LoRA

- **Particle** = one rank-1 component per layer: $\theta_i = (\mathbf{a}_i \,\|\, \mathbf{b}_i) \in \mathbb{R}^{d_{\text{in}}+d_{\text{out}}}$
- **Kernel**: Gaussian (RBF), length-scale by the median heuristic

$$\ell^2 = \tfrac{1}{2}\,\text{median}\{\|\theta_i - \theta_j\|^2\}$$

- **Optimiser**: Adam on the VGD direction, as is standard for SVGD
- Same data, schedule and learning rate as the baselines

Cost

Kernel term: $O(m^2 p)$ per step. Small next to the frozen 8B forward–backward pass.

Method	Wall-clock (8×H100)
Standard LoRA	∼5h 30m
MFLD	∼5h 30m
VGD	∼5h 50m (+6%)

$m = 32$, $r = 1$, rank 32, 3 epochs

A design choice that matters: the joint kernel

How do we measure the distance between two rank-1 components?

Separate kernel

$$k_A(\mathbf{a}_i, \mathbf{a}_j) + k_B(\mathbf{b}_i, \mathbf{b}_j)$$

Joint kernel

$$k\big((\mathbf{a}_i \| \mathbf{b}_i), (\mathbf{a}_j \| \mathbf{b}_j)\big)$$

The joint kernel keeps the cross-terms and better reflects the distance between $\mathbf{b}_i\mathbf{a}_i^\top$ and $\mathbf{b}_j\mathbf{a}_j^\top$. Same cost.

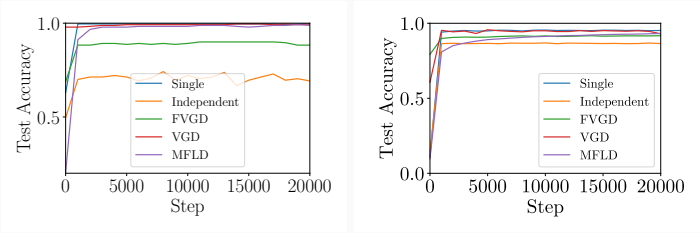
Average accuracy (%), rank 32

	8B	3B	1B
MFLD	85.19	80.51	65.54
VGD, joint	**85.42**	**80.94**	**66.34**
VGD, separate	83.96	74.11	55.41

Separate kernel costs -1.2, -6.4, -10.1 points vs MFLD.

Results

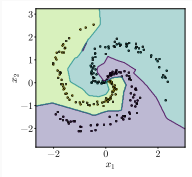
Sanity check first: ensembles on toy tasks



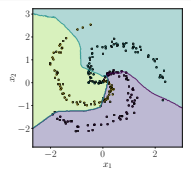
Test accuracy. Left: Spiral. Right: MNIST.

- 10 deliberately **tiny** MLPs (hidden width 2 or 4)
- Independent training is clearly worst
- Joint training (MFLD, VGD) matches **one larger network**
- MNIST: VGD 0.948, MFLD 0.932

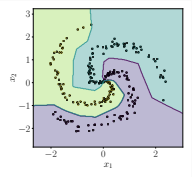
Spiral: what the combined models learn



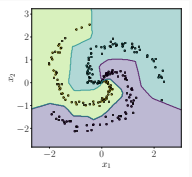
Single (width 20)



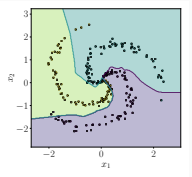
Independent



MFLD



VGD



FVGD

- Each ensemble member has hidden width 2: it **cannot** solve the task alone
- Trained jointly, 10 weak models recover the spiral; trained independently, they do not

LLM set-up

Data and evaluation

- Train: Commonsense-170K [Hu et al., 2023]
- Test: 8 benchmarks
  - BoolQ, PIQA, SIQA, HellaSwag
  - WinoGrande, ARC-e, ARC-c, OBQA
- Metric: accuracy; we report the 8-task average

Training

- Llama-3-8B (also 3B, 1B), FP16
- 3 epochs, batch 16, lr 10^{-4}, AdamW
- Protocol follows Nitanda et al. (2025)
- λ: MFLD 10^{-5}; VGD 10^{-7} (rank 32), 10^{-6} (rank 256)

Standard = one AdamW-trained LoRA of the same total rank.

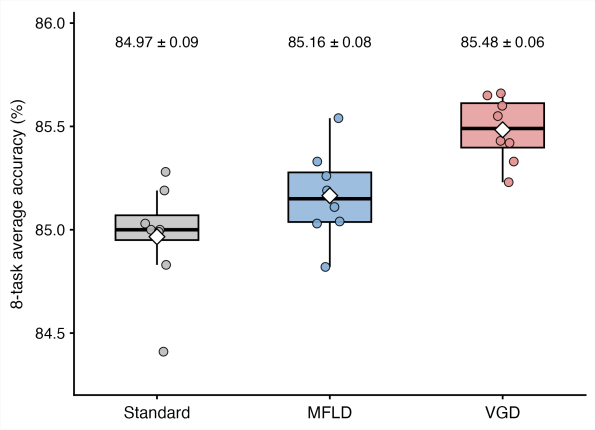
Result 1: single rank-32 adapter (8 seeds)

Method	SIQA	PIQA	Wino	OBQA	ARC-c	ARC-e	BoolQ	Hella	**Avg**
Standard	80.23	88.38	86.28	85.67	78.83	90.26	74.82	95.26	84.97
MFLD	80.29	88.28	86.39	86.09	**79.83**	90.26	74.95	95.22	85.16
VGD	**80.86**	**88.86**	**86.48**	**86.64**	**79.83**	**90.75**	**75.00**	**95.44**	**85.48**

- VGD is best or tied-best on **all 8** tasks
- Paired bootstrap over tasks:
  - VGD $-$ Standard: $+0.52$ $[+0.31, +0.74]$
  - VGD $-$ MFLD: $+0.32$ $[+0.16, +0.48]$

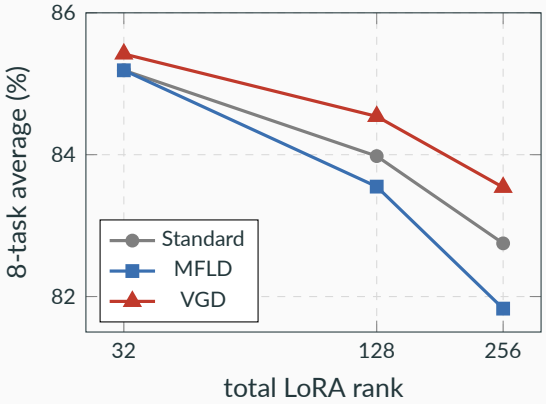
Gains are modest at rank 32, but consistent and significant.

Result 1 (cont.): VGD is also more stable across seeds



- Each dot = one training run
- Medians: 85.00 / 85.15 / 85.49
- VGD: highest median, smallest standard error, no bad seed
- Best single run: 85.66 (VGD)

Result 2: at high rank, VGD helps most



single-seed λ sweep, best λ for VGD

Single rank-256 adapter

Method	Avg
Standard	82.75
MFLD	81.83
VGD	**83.54**

- VGD − MFLD: $+1.71$ $[+1.38, +2.04]$
- VGD − Standard: $+0.79$ $[+0.38, +1.24]$
- The noise of MFLD **hurts** at high rank

Where does the gain come from?

Rank 256, VGD, single seed

Setting	Avg (%)
Standard ($\lambda = 0$)	82.75
VGD, $\lambda = 0$	82.92
VGD, $\lambda = 10^{-7}$	83.05
VGD, $\lambda = 3.2 \times 10^{-7}$	83.30
VGD, $\lambda = 10^{-6}$	**83.54**
VGD, $\lambda = 10^{-5}$	78.47

1. **Kernel coupling** alone helps: $82.92 > 82.75$
2. **Entropy / repulsion** adds more: best at $\lambda = 10^{-6}$
3. Too much repulsion is harmful: performance drops fast for $\lambda \geq 10^{-5}$

λ needs tuning, and the best value depends on model size.

Result 3: merging many adapters (PoC merge)

8 independent rank-32 adapters → one rank-256 adapter (same test cost as rank-256 LoRA)

Method	SIQA	PIQA	Wino	OBQA	ARC-c	ARC-e	BoolQ	Hella	**Avg**
Standard	82.12	89.74	88.48	88.24	82.91	92.16	76.05	96.27	87.00
MFLD	82.18	**90.14**	88.73	88.08	**83.35**	92.17	76.25	96.22	87.14
VGD	**82.55**	90.10	**88.97**	**88.49**	83.18	**92.36**	**76.39**	**96.41**	**87.31**

- Merging helps every method: about $+2$ **points** over its single rank-32 adapter
- After merging, VGD is still best: $+0.31$ $[+0.24, +0.39]$ over Standard, $+0.17$ $[+0.04, +0.28]$ over MFLD
- Compare with a single rank-256 adapter: 87.31 vs 82.75

Result 4: truly joint training of an ensemble

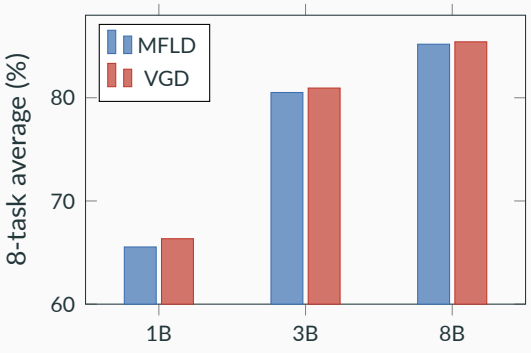
- PoC merge trains its 8 adapters **independently**
- Now: 8 rank-32 LoRA branches, **jointly** trained as one ensemble (logit averaging)
- Higher cost: 8× training and 8× inference

Method	Avg (%)
PoC merge, MFLD	87.14
PoC merge, VGD	87.31
Joint ensemble, MFLD	89.14
Joint ensemble, VGD	**89.33**

Take-away

Joint training adds about +2 **more points** on top of merging. Training the models together, not separately, pays off.

Result 5: other model sizes



Llama family, rank 32, single seed

- VGD $\geq$ MFLD at every size
- Margin: $+0.23$ (8B), $+0.43$ (3B), $+0.80$ (1B)
- Best λ: 10^{-7} (8B), 10^{-6} (3B), 10^{-5} (1B)

Single seed only: margins are small and need multi-seed replication.

Results summary

Setting	Standard	MFLD	VGD
Single rank-32 adapter (8 seeds)	84.97	85.16	**85.48**
Single rank-256 adapter	82.75	81.83	**83.54**
PoC merge $8 \times 32 \to 256$	87.00	87.14	**87.31**
Joint ensemble, $8\times$ rank 32	—	89.14	**89.33**

1. VGD matches or beats MFLD in every setting, at $\sim$6% extra cost
2. The gain is largest at high rank, where MFLD's noise hurts
3. Many small adapters $\gg$ one big adapter; joint training $>$ independent training

A Negative Result: Functional VGD

Functional VGD (FVGD): the idea

Worry: kernels on high-dimensional parameters behave badly [Ba et al., 2021].

Idea: put the kernel on model **outputs** [Wang et al., 2019; D'Angelo & Fortuin, 2021]

1. Represent each model by its outputs on a batch: $F_n(\theta) \in \mathbb{R}^{ne}$
2. Compute the VGD direction ϕ in function space
3. Pull it back with the Jacobian $J_i = D_\theta F_n(\theta_i)$:

$$\theta_i^{t+1} = \theta_i^t + \epsilon J_i^\top \phi_i$$

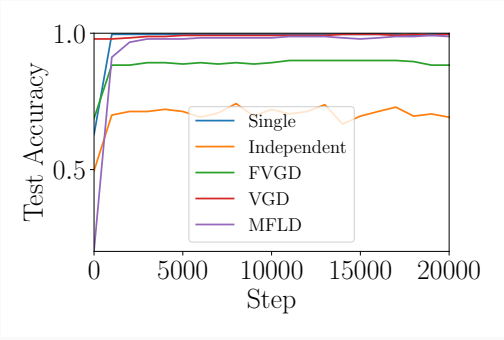
Our contribution here

- Extend the idea from Bayesian posteriors to a **general** loss $\mathcal{L}$
- Test it carefully

Limitation

FVGD works for ensembles only. It does **not** apply to LoRA averaging: there the averaging happens *before* the non-linearity.

FVGD did not work



Spiral, test accuracy: FVGD (green) is below MFLD and VGD

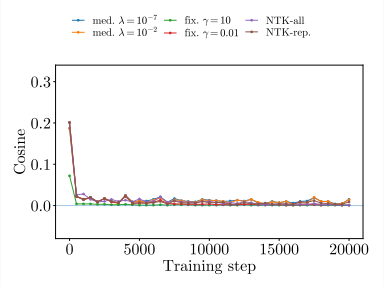
- **Spiral:** worse than MFLD and VGD
- **MNIST:** test accuracy 0.918–0.923, vs MFLD 0.932 and VGD 0.948
- **Large λ:** FVGD collapses (accuracy 0.114 for $\lambda \geq 1$), while VGD stays at 0.948 for all $\lambda \in [0, 20]$

No consistent benefit from working in function space.

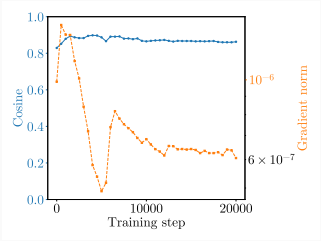
Why it failed: the Jacobian pull-back

To first order, the actual change in function space is

$$F_n(\theta_i^{t+1}) - F_n(\theta_i^t) \approx \epsilon \underbrace{J_i J_i^\top}_{\text{tangent kernel}} \phi_i \neq \epsilon \phi_i.$$



$\cos(JJ^\top \phi, \phi) \approx 0$



inverse pull-back: aligned but tiny

- Raw pull-back: cosine only 0.002–0.012
- Inverse pull-back: cosine ≈ 0.875, but norm 7.1×10^{-7} vs gradient 0.114
- Repulsion is **lost**

Conclusion and Future Work

Future work

1. **Better geometry.** VGD uses Stein geometry. Wasserstein-based alternatives may scale better in high dimensions.
2. **Fix the pull-back.** Find a parameter update that really follows the function-space direction, then bring FVGD to LLM fine-tuning.
3. **Cheaper joint training.** The joint ensemble gives the best accuracy but costs $8\times$. Can we get it at $1\times$?
4. **Choosing λ automatically.** The best λ moves with model size and rank.
5. **Harder tasks.** Generation, mathematical reasoning, other model families, and multi-seed studies at scale.

Take-home messages

1. **Many LoRAs beat one big LoRA.** Rank-256 standard: 82.75. Merged 8×32: 87.31. Joint ensemble: 89.33.
2. **Treat the adapters as a distribution.** Joint training becomes $\min_Q \mathcal{L}(Q) + \lambda \mathcal{E}(Q)$.
3. **Ensembles $\neq$ LoRA averaging.** One is convex, one is not.
4. **VGD $>$ MFLD.** Shared gradients plus deterministic repulsion beat gradient plus noise, at $\sim$6% extra cost.
5. **Function space is not free.** FVGD failed, and we know why.

Thank you!

Questions?

A Distributional Optimisation Perspective on Combining Models in Deep Learning
arXiv:2609.24328

Backup: entropy vs KL regularisation

For a confining potential V with $Q_0 \propto e^{-V/\lambda}$,

$$\mathcal{L}(Q) + \lambda\,\mathrm{KL}(Q \,\|\, Q_0) = \underbrace{\mathcal{L}(Q) + \int V \,\mathrm{d}Q}_{\tilde{\mathcal{L}}(Q)} + \lambda\,\mathcal{E}(Q) + \text{const.}$$

- KL regularisation $=$ entropy regularisation with a modified loss
- So theory written for KL (e.g. Chazal et al., 2025) applies to our objective

Backup: λ sensitivity

MNIST ensembles (test accuracy)

- VGD: 0.948 for all $\lambda \in [0, 20]$
- MFLD: $[0.926, 0.932]$
- FVGD: $[0.918, 0.923]$ for $\lambda \leq 10^{-3}$, collapses for $\lambda \geq 1$

LLM, VGD, rank 32

- $\lambda = 10^{-7}$: 85.42
- $\lambda = 10^{-5}$: 84.62
- $\lambda = 10^{-4}$: 74.28 (-11.14)

Model size (best λ)

- 8B: 10^{-7}, 3B: 10^{-6}, 1B: 10^{-5}